

Image Classification

Chapter Goals

After completing this chapter, you should be able to understand :

- What is image classification
- How to build features for image classification
- Limitations of traditional approaches for image classification
- How to implement an image classification pipeline in python

Image classification

Classification

Image Classification: A core task in Computer Vision



This image by Nikita is
licensed under CC-BY 2.0

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}



cat

Classification



Classification: Important for other tasks

Example: Object Detection



Classification: Important for other tasks

Example: Object Detection



Background

Donut

→ Coffee

Person

Car

Classification: Important for other tasks

Example: Object Detection



Background
Donut
Coffee
Person
Car

Data Classification



Label = **F**(features)

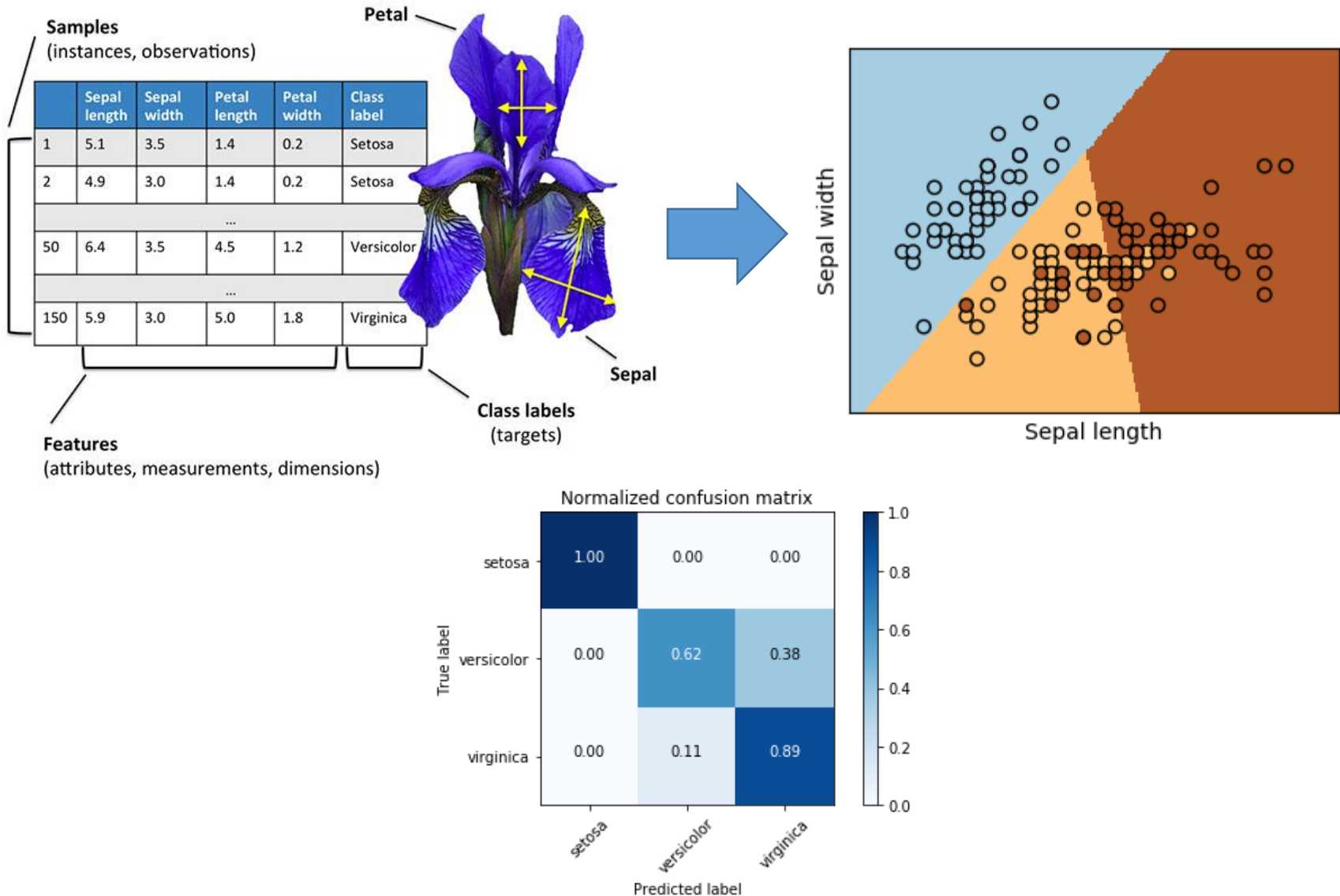
Versicolor (0)

Setosa (1)

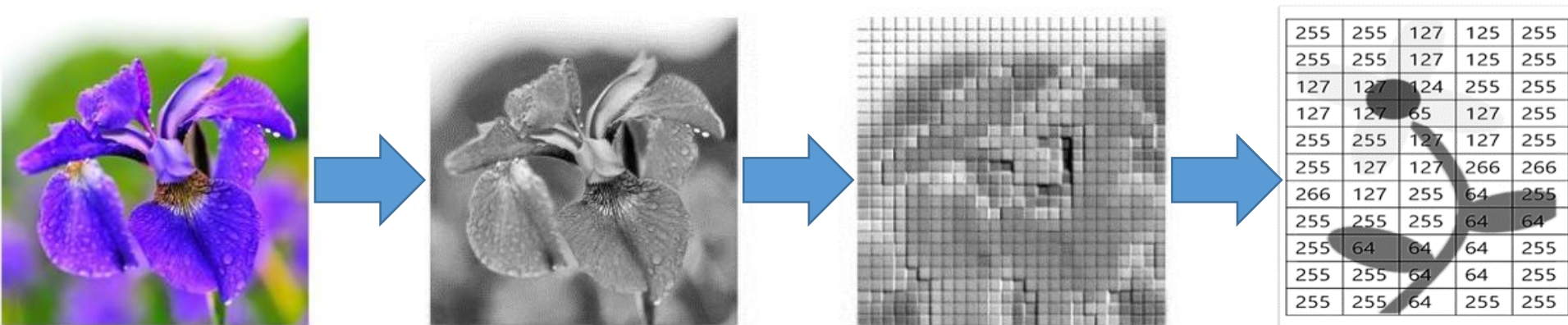
Virginica (2)

- Sepal length
- Sepal width
- Petal length
- Petal width

Data Classification



Classical image classification



Train

```
[255,255,127,255,255,255,255,127,125,255...] [ 0
[255,255,123,225,207,255,205,127,120,255...] 1
[251,225,127,255,255,255,255,130,135,255...] 0
[255,255,127,255,235,255,255,127,125,255...] 2
[255,205,127,255,255,255,222,127,125,255...] ] 0]
```

Validate

```
[255,255,124,255,251,255,255,127,125,255...] [ 1
[255,205,127,255,255,255,255,137,125,255...] 0
[255,255,127,235,255,255,255,129,125,255...] 2
[255,255,125,255,255,255,255,127,125,255...] ] 0]
```


Classical image classification



Trained model



Label: *Virginica*

Classical image classification: Features



Trained model



Label: Virginica

Label= F (features)



No obvious way to choose features

Classical image classification: Features

We could try:



or

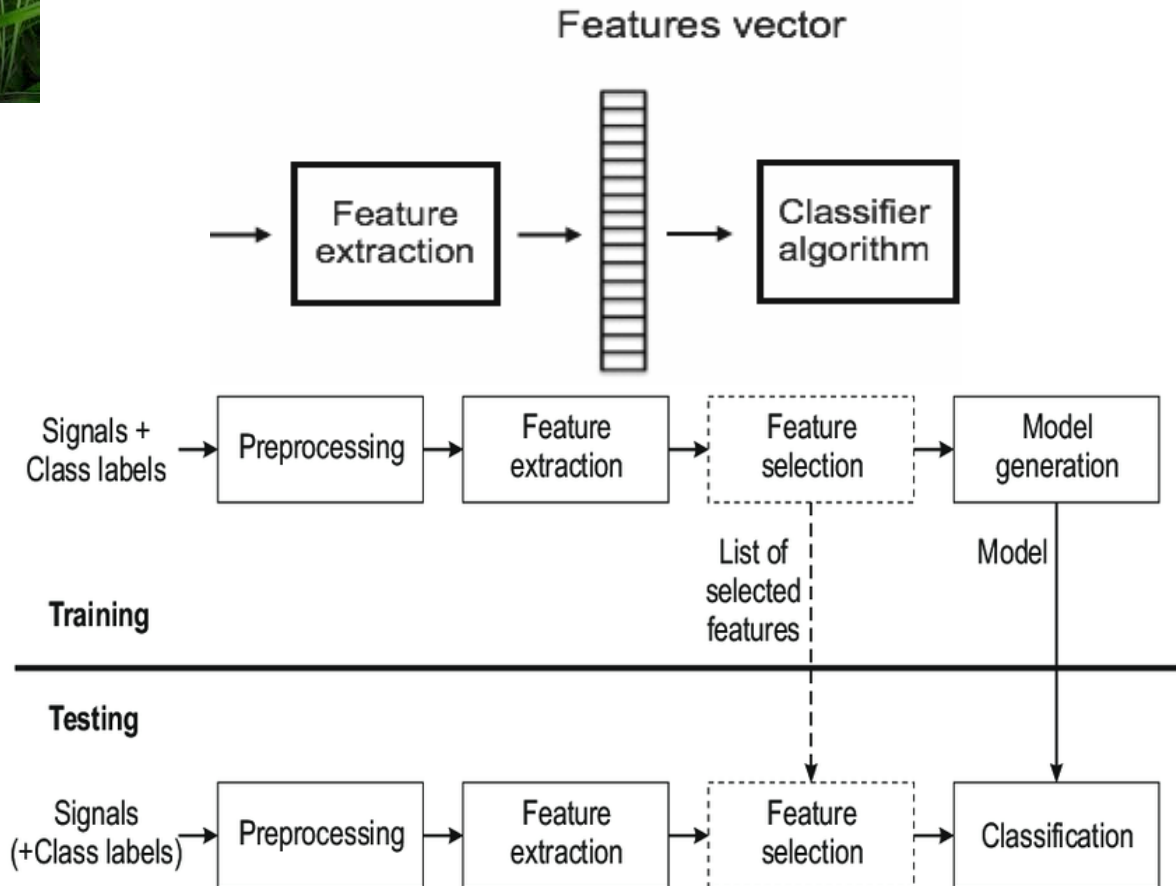


Or...

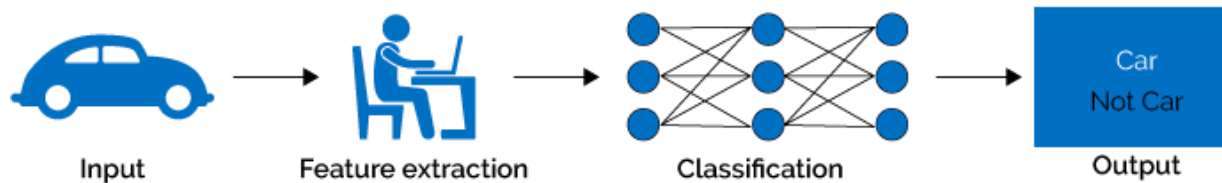


Label: Versicolor

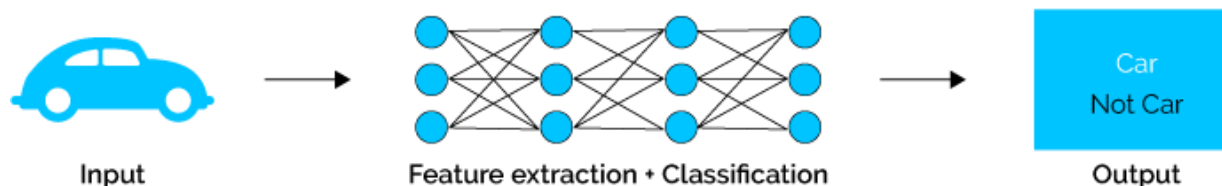
Classical image classification: Features



Machine Learning



Deep Learning



Deep Learning Vs Machine Learning

Factors

Data Requirement

Accuracy

Training Time

Hardware Dependency

Hyperparameter Tuning

Deep Learning

Requires large data

Provides high accuracy

Takes longer to train

Requires GPU to train properly

Can be tuned in various different ways.

Machine Learning

Can train on lesser data

Gives lesser accuracy

Takes less time to train

Trains on CPU

Limited tuning capabilities

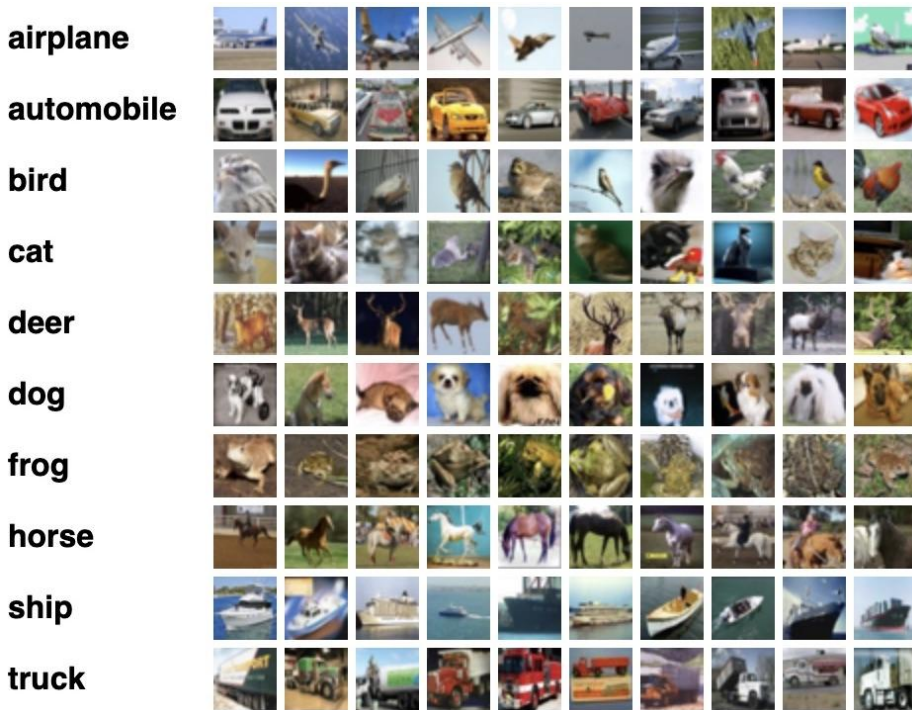
Image Classification Datasets: MNIST



10 classes: Digits 0 to 9
28x28 grayscale images
50k training images
10k test images

Results from MNIST often do not hold on more complex datasets

Image Classification Datasets: CIFAR10



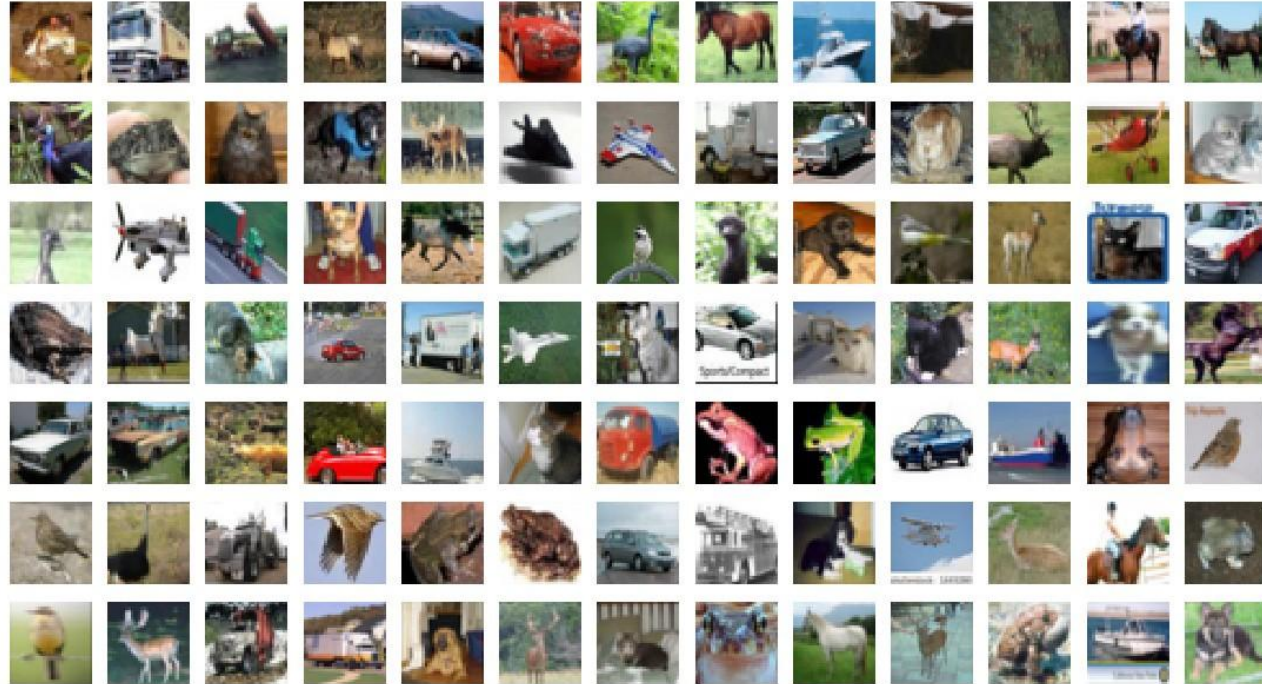
10 classes

50k training images (5k per class)

10k testing images (1k per class)

32x32 RGB images

Image Classification Datasets: CIFAR100



100 classes

50k training images (500 per class)

10k testing images (100 per class)

32x32 RGB images

20 superclasses with 5 classes each:

Aquatic mammals: beaver, dolphin, otter, seal, whale

Trees: Maple, oak, palm, pine, willow

Image Classification Datasets: ImageNet



1000 classes

~1.3M training images (~1.3K per class)

50K validation images (50 per class)

...100K test images (100 per class)

Performance metric: Top 5 accuracy

Algorithm predicts 5 labels for each image;

...one of them needs to be right

Images have variable size,

but often resized to 256x256 for training

Model performance

Accuracy Ratio of correctly predicted observation to the total observations. Accuracy is a good measure but only when values of false positive and false negatives are almost same. Therefore, you have to look at other parameters to evaluate the performance of your model.

$$\text{Accuracy} = \frac{TP+TN}{TP+FP+FN+TN}$$

Precision Ratio of correctly predicted positive observations to the total predicted positive observations. High Precision relates to the low false positive rate.

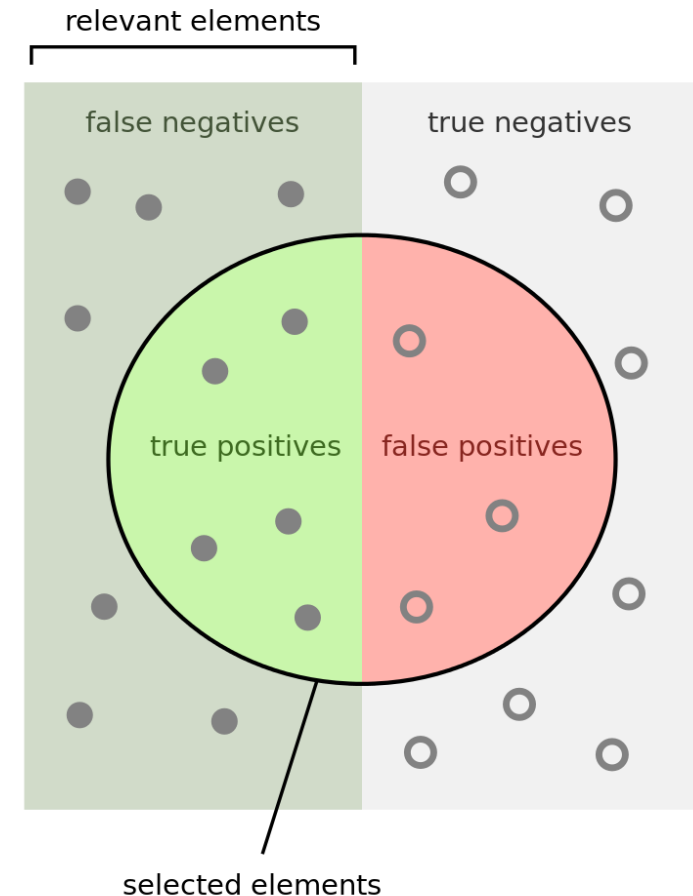
$$\text{Precision} = \frac{TP}{TP+FP}$$

Recall Ratio of correctly predicted positive observations to the all observations in actual class

$$\text{Recall} = \frac{TP}{TP+FN}$$

F1 score - F1 Score is the weighted average of Precision and Recall. Therefore, this score takes both false positives and false negatives into account.

$$\text{F1 Score} = \frac{2 * (\text{Recall} * \text{Precision})}{(\text{Recall} + \text{Precision})}$$



How many selected items are relevant?

$$\text{Precision} = \frac{\text{Green Circle}}{\text{Green Circle} + \text{Red Circle}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{Green Circle}}{\text{Green Circle} + \text{Dark Green Circle}}$$

Limitations to classical image classification

- Algorithms are susceptible to variations in color, contrast, lighting, size, angle...
- This can be mitigated by image preprocessing, but can require a lot of work
- Techniques apply to entire array of pixels, but only some pixels describe the object you are trying to classify
- Can be heavily influenced by background pixels that have little or no effect on the class of object

The basic concepts of Neural Networks

What is Deep Learning?

ARTIFICIAL INTELLIGENCE

Any technique that enables computers to mimic human behavior



MACHINE LEARNING

Ability to learn without explicitly being programmed



DEEP LEARNING

Extract patterns from data using neural networks



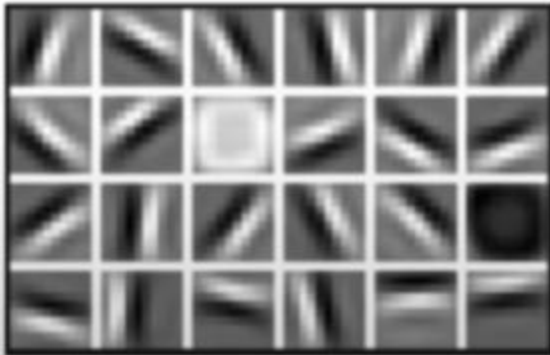
Teaching computers how to **learn a task** directly from **raw data**

Why Deep Learning, and Why Now?

Hand engineered features are time consuming, brittle, and not scalable in practice

Can we learn the **underlying features** directly from data?

Low Level Features



Lines & Edges

Mid Level Features



Eyes & Nose & Ears

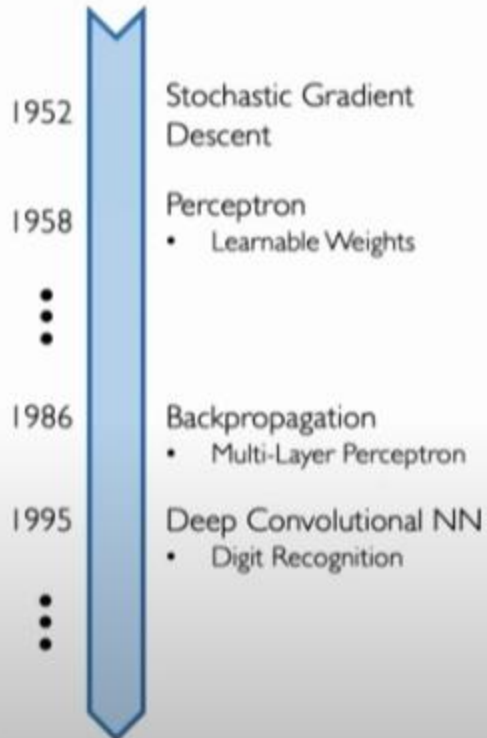
High Level Features



Facial Structure

Why Now?

Neural Networks date back decades, so why the resurgence?



1. Big Data

- Larger Datasets
- Easier Collection & Storage

IMAGENET



2. Hardware

- Graphics Processing Units (GPUs)
- Massively Parallelizable



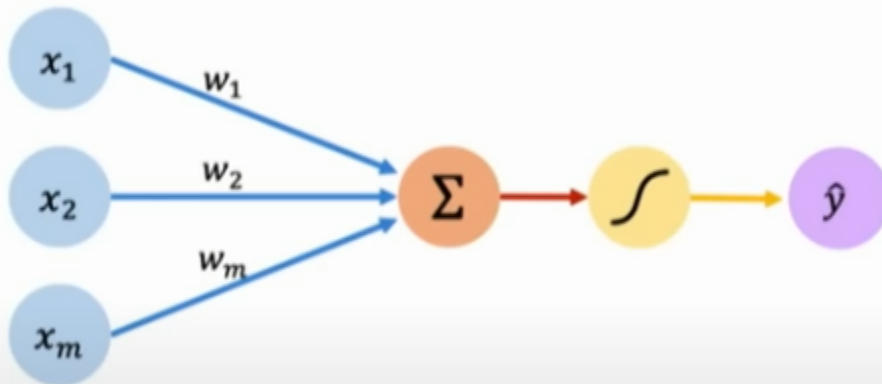
3. Software

- Improved Techniques
- New Models
- Toolboxes



The perceptron. The structural building block of deep learning

The Perceptron: Forward Propagation



Output

Linear combination of inputs

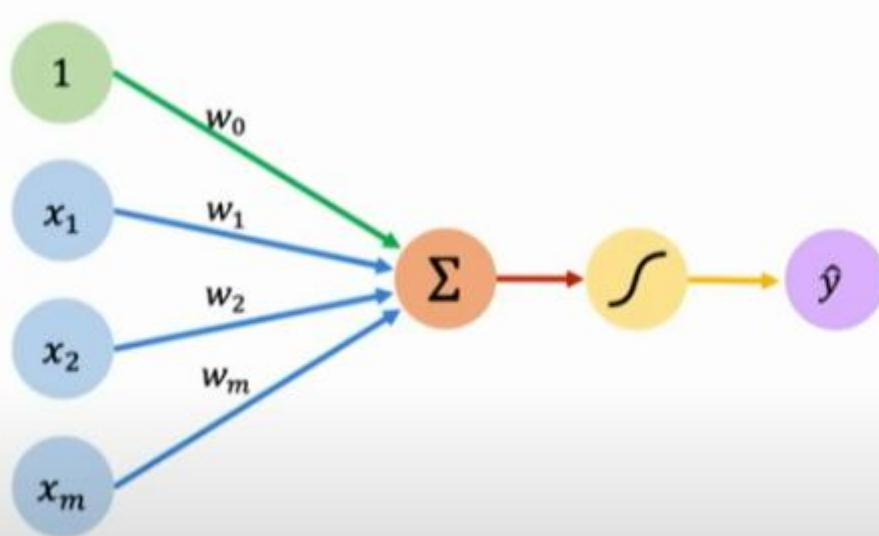
$$\hat{y} = g \left(\sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

Inputs Weights Sum Non-Linearity Output

The perceptron. The structural building block of deep learning

The Perceptron: Forward Propagation



Inputs Weights Sum Non-Linearity Output

Output

Linear combination of inputs

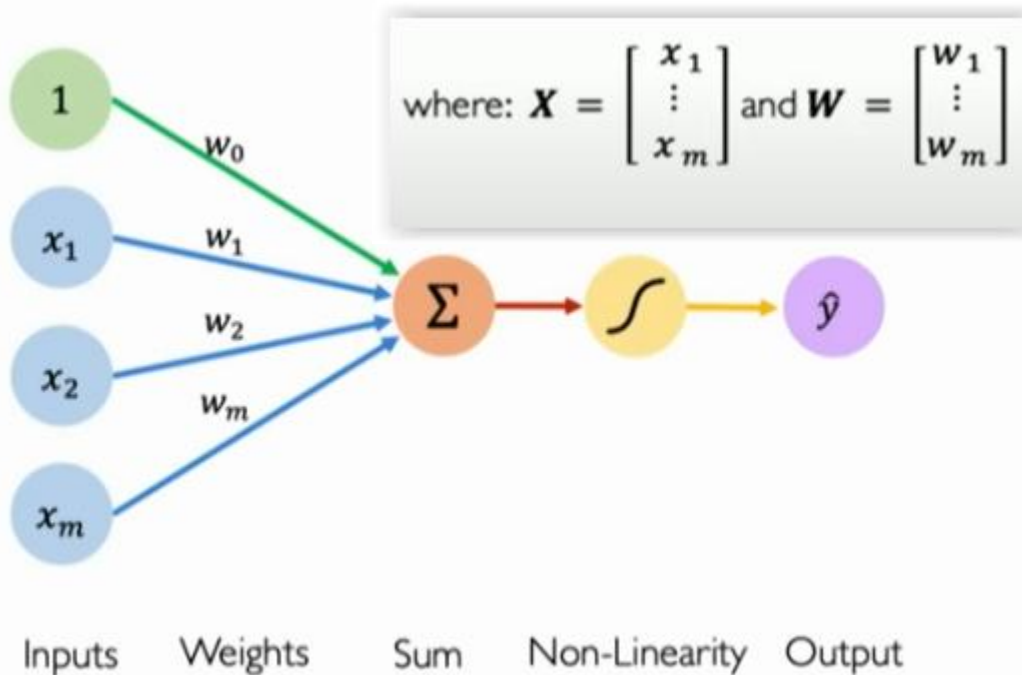
$$\hat{y} = g \left(w_0 + \sum_{i=1}^m x_i w_i \right)$$

Non-linear activation function

Bias

The perceptron. The structural building block of deep learning

The Perceptron: Forward Propagation

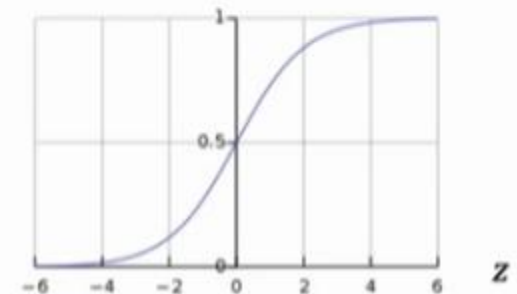


Activation Functions

$$\hat{y} = g(w_0 + \mathbf{X}^T \mathbf{W})$$

- Example: sigmoid function

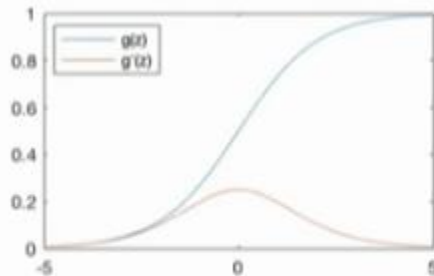
$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



The perceptron. The structural building block of deep learning

Common Activation Functions

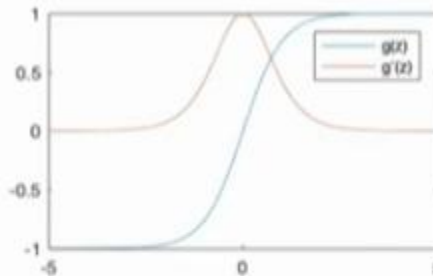
Sigmoid Function



$$g(z) = \frac{1}{1 + e^{-z}}$$

$$g'(z) = g(z)(1 - g(z))$$

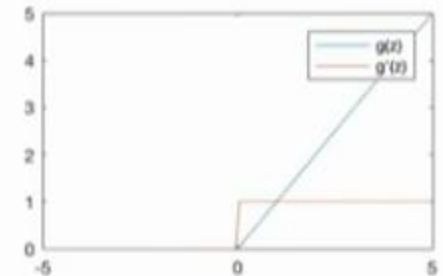
Hyperbolic Tangent



$$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$g'(z) = 1 - g(z)^2$$

Rectified Linear Unit (ReLU)



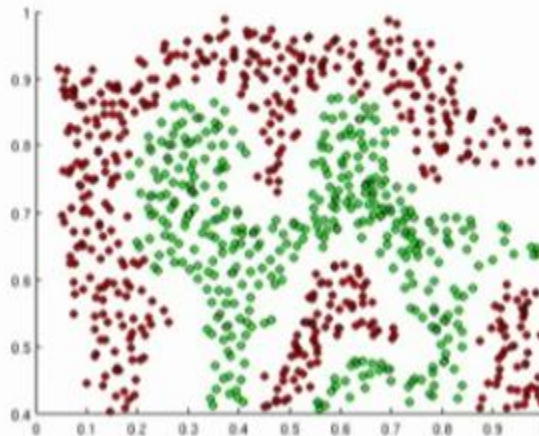
$$g(z) = \max(0, z)$$

$$g'(z) = \begin{cases} 1, & z > 0 \\ 0, & \text{otherwise} \end{cases}$$

The perceptron. The structural building block of deep learning

Importance of Activation Functions

*The purpose of activation functions is to **introduce non-linearities** into the network*

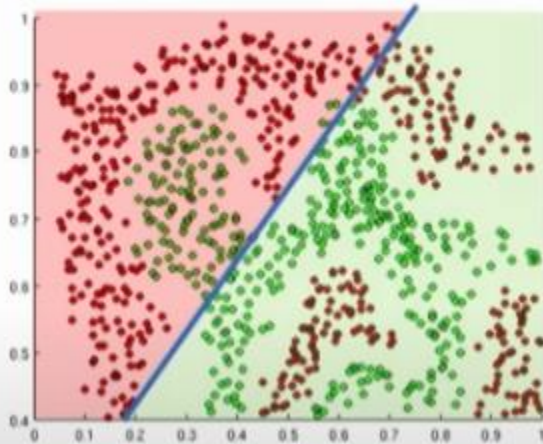


What if we wanted to build a neural network to distinguish green vs red points?

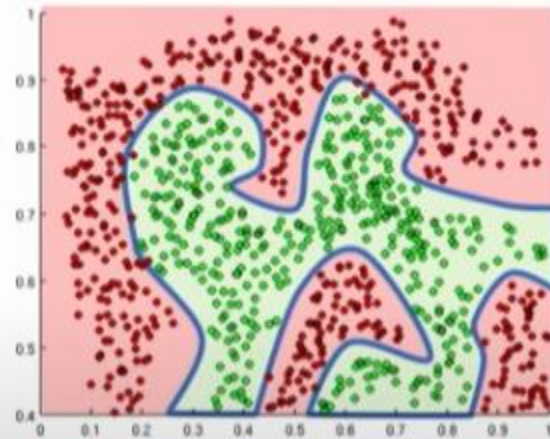
The perceptron. The structural building block of deep learning

Importance of Activation Functions

*The purpose of activation functions is to **introduce non-linearities** into the network*



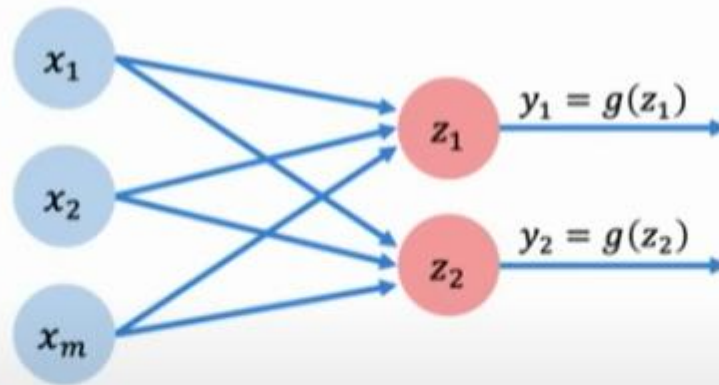
Linear activation functions produce linear decisions no matter the network size



Non-linearities allow us to approximate arbitrarily complex functions

Building Neural Networks With Perceptrons

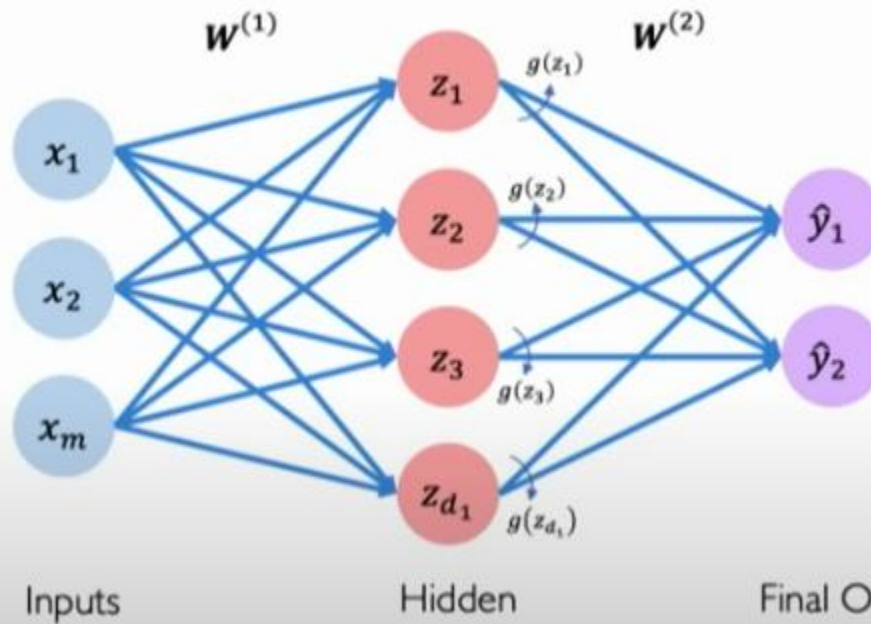
Multi Output Perceptron



$$z_i = w_{0,i} + \sum_{j=1}^m x_j w_{j,i}$$

Building Neural Networks With Perceptrons

Single Layer Neural Network

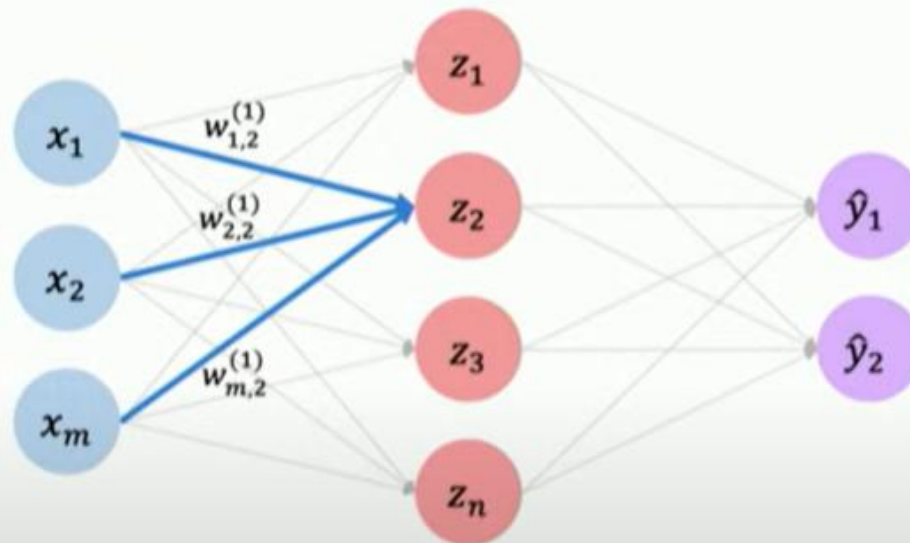


$$z_l = w_{0,l}^{(1)} + \sum_{j=1}^m x_j w_{j,l}^{(1)}$$

$$\hat{y}_l = g \left(w_{0,l}^{(2)} + \sum_{j=1}^{d_1} g(z_j) w_{j,l}^{(2)} \right)$$

Building Neural Networks With Perceptrons

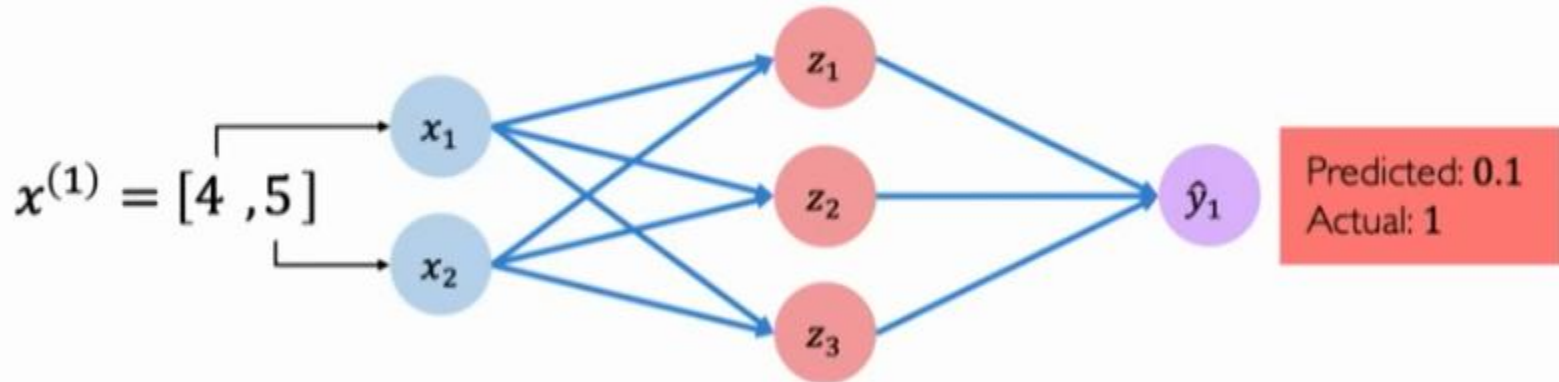
Single Layer Neural Network



$$\begin{aligned} z_2 &= w_{0,2}^{(1)} + \sum_{j=1}^m x_j w_{j,2}^{(1)} \\ &= w_{0,2}^{(1)} + x_1 w_{1,2}^{(1)} + x_2 w_{2,2}^{(1)} + x_m w_{m,2}^{(1)} \end{aligned}$$

Quantifying Loss

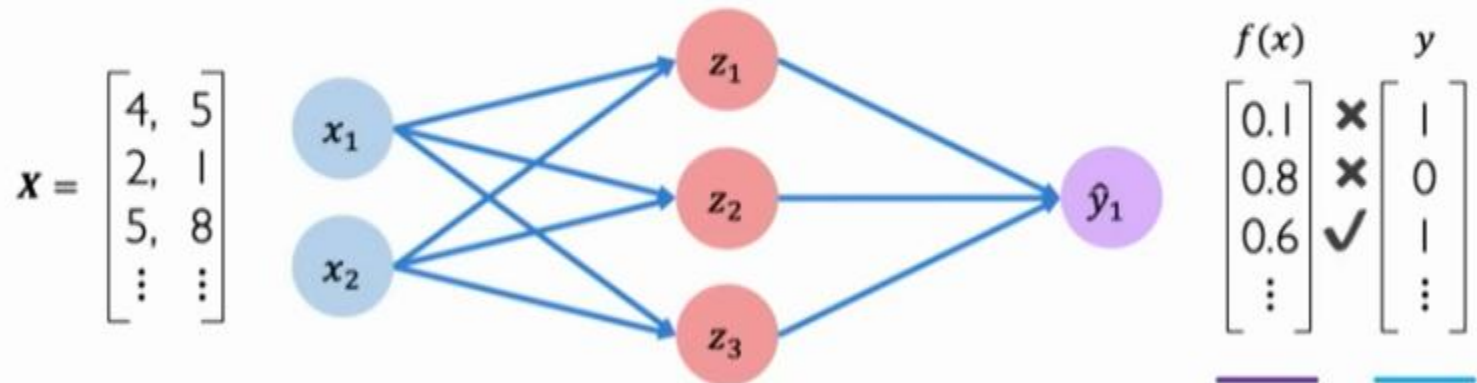
The **loss** of our network measures the cost incurred from incorrect predictions



$$\mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Empirical Loss

The **empirical loss** measures the total loss over our entire dataset



Also known as:

- Objective function
- Cost function
- Empirical Risk

$$J(\mathbf{W}) = \frac{1}{n} \sum_{i=1}^n \mathcal{L}(\underbrace{f(x^{(i)}; \mathbf{W})}_{\text{Predicted}}, \underbrace{y^{(i)}}_{\text{Actual}})$$

Training A Neural Network

Loss Optimization

We want to find the network weights that *achieve the lowest loss*

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} \frac{1}{n} \sum_{i=1}^n \mathcal{L}(f(x^{(i)}; \mathbf{W}), y^{(i)})$$

$$\mathbf{W}^* = \operatorname{argmin}_{\mathbf{W}} J(\mathbf{W})$$

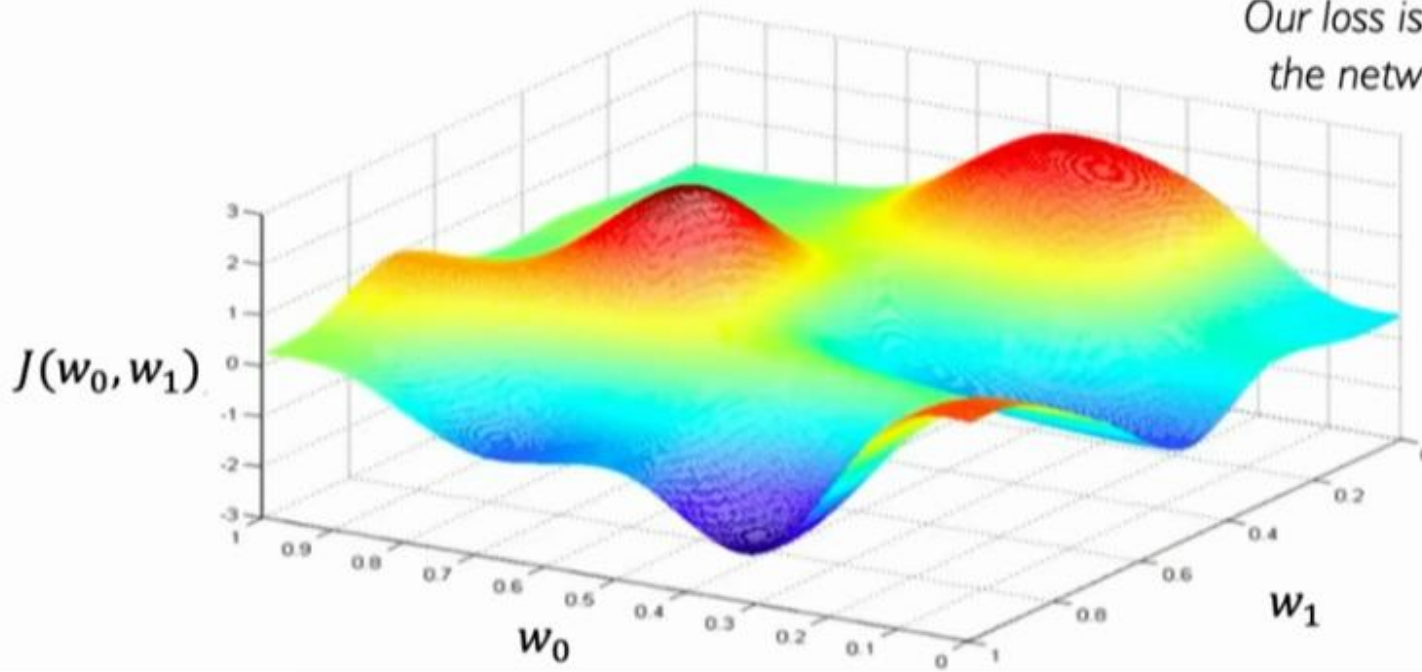
Remember:

$$\mathbf{W} = \{\mathbf{W}^{(0)}, \mathbf{W}^{(1)}, \dots\}$$

Loss Optimization

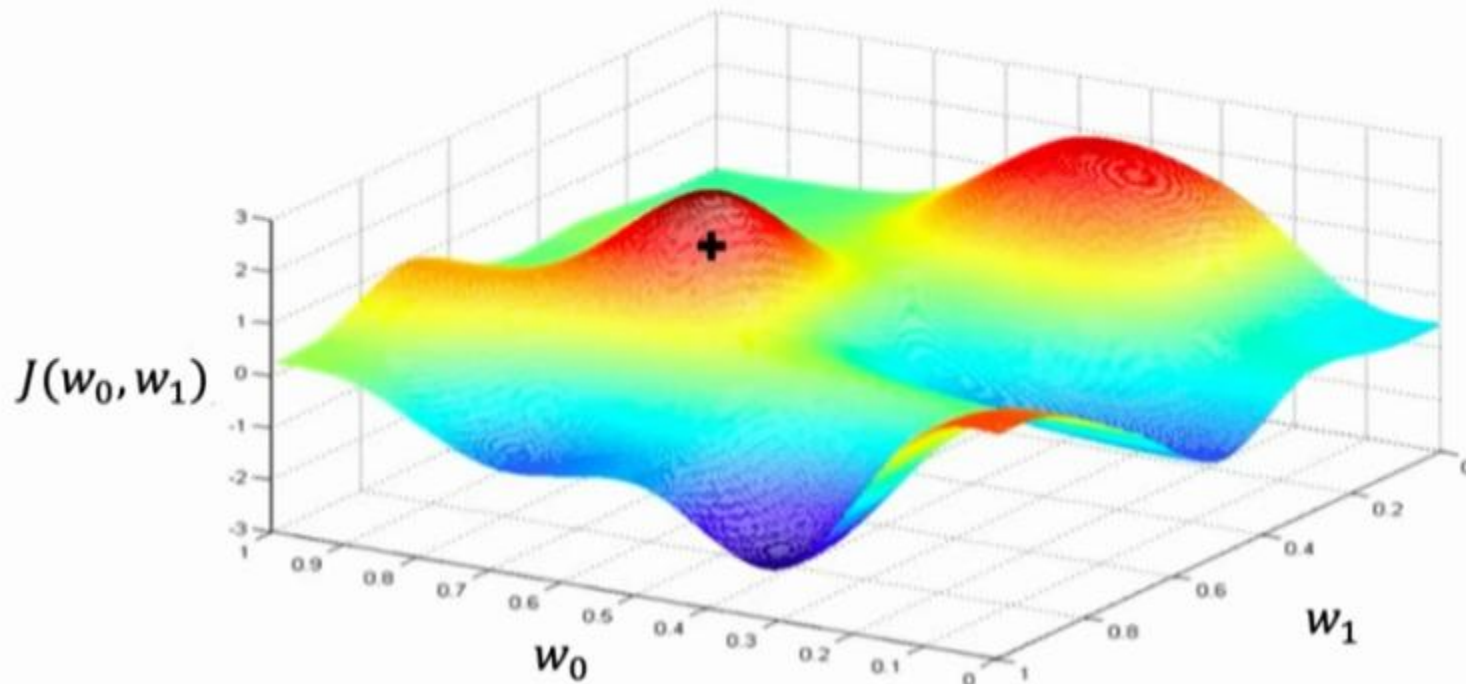
$$W^* = \operatorname{argmin}_W J(W)$$

Remember:
*Our loss is a function of
the network weights!*



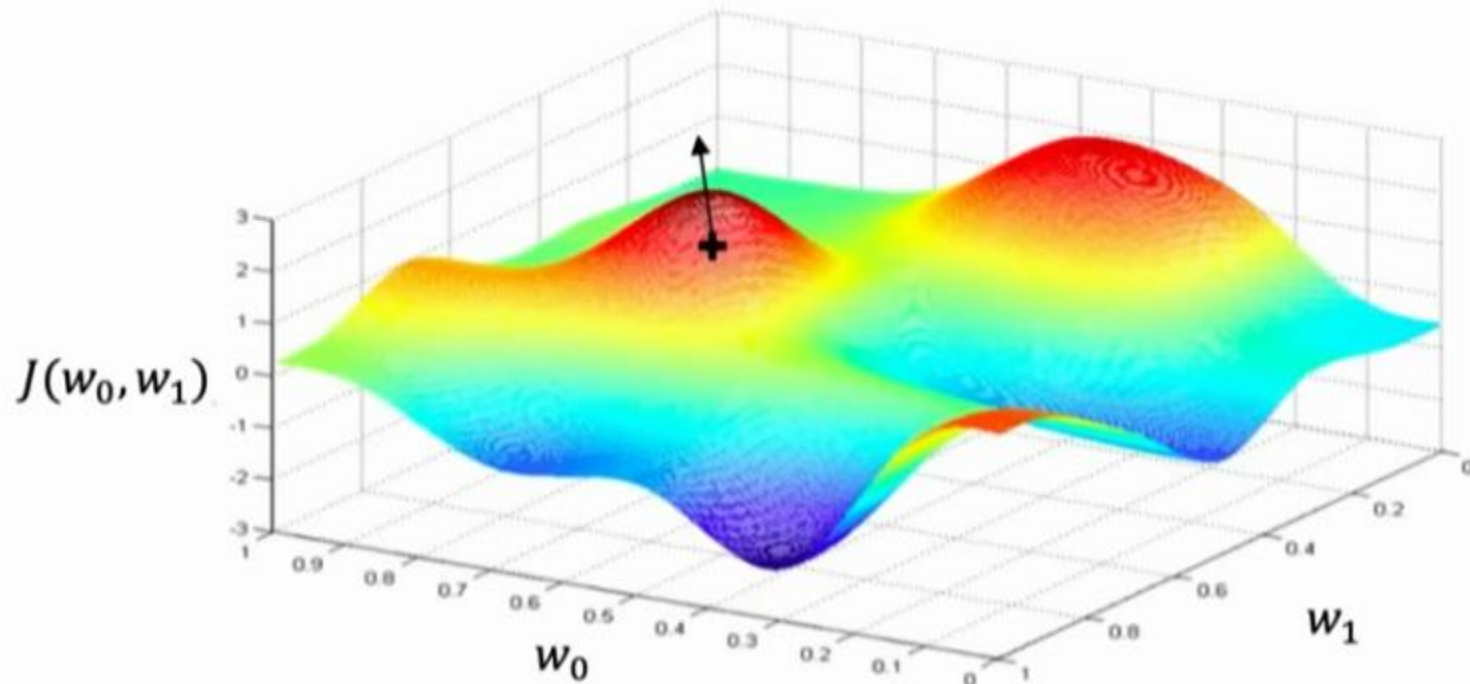
Loss Optimization

Randomly pick an initial (w_0, w_1)



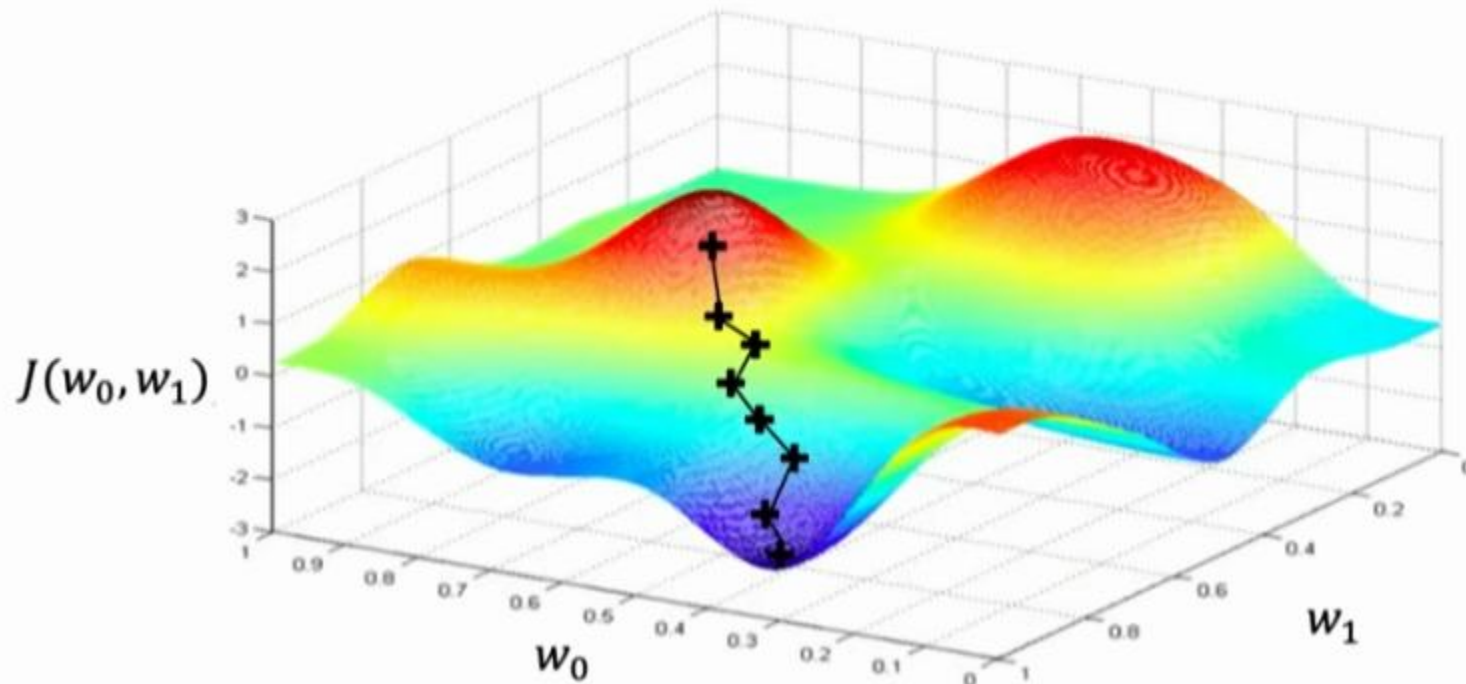
Loss Optimization

Compute gradient, $\frac{\partial J(w)}{\partial w}$



Gradient Descent

Repeat until convergence



Gradient Descent

Algorithm

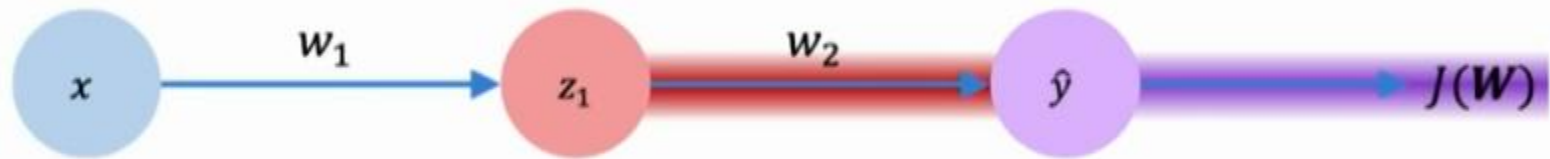
1. Initialize weights randomly $\sim \mathcal{N}(0, \sigma^2)$
2. Loop until convergence:
3. Compute gradient, $\frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
4. Update weights, $\mathbf{W} \leftarrow \mathbf{W} - \eta \frac{\partial J(\mathbf{W})}{\partial \mathbf{W}}$
5. Return weights

Computing Gradients: Backpropagation



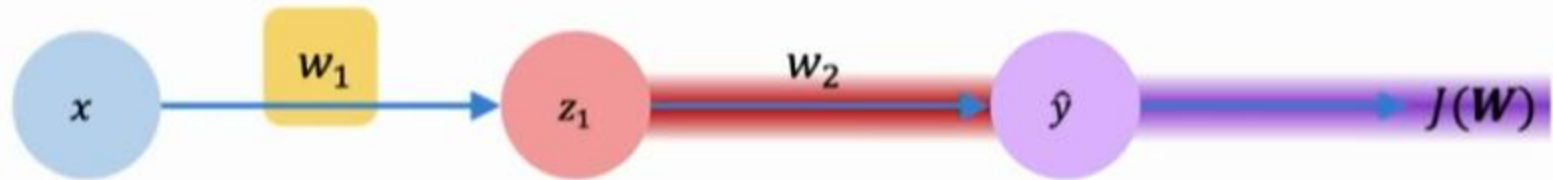
How does a small change in one weight (ex. w_2) affect the final loss $J(\mathbf{W})$?

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_2} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial w_2}}_{\text{red}}$$

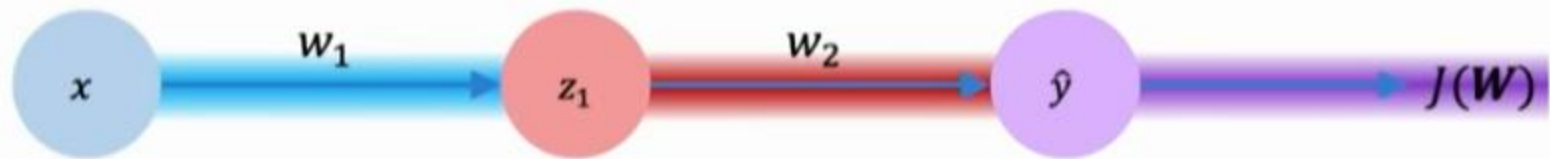
Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_1} = \frac{\partial J(W)}{\partial \hat{y}} * \frac{\partial \hat{y}}{\partial w_1}$$

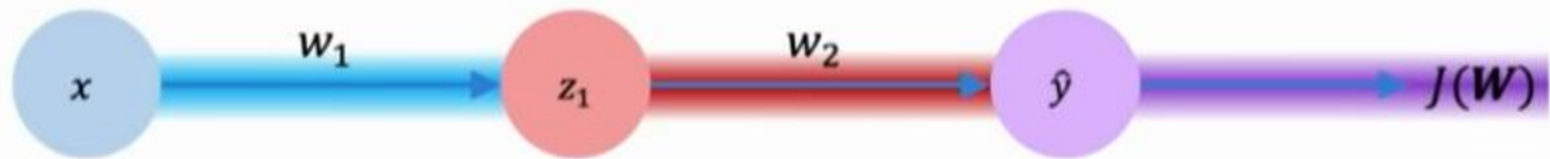
Apply chain rule! Apply chain rule!

Computing Gradients: Backpropagation



$$\frac{\partial J(\mathbf{W})}{\partial w_1} = \underbrace{\frac{\partial J(\mathbf{W})}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Computing Gradients: Backpropagation



$$\frac{\partial J(W)}{\partial w_1} = \underbrace{\frac{\partial J(W)}{\partial \hat{y}}}_{\text{purple}} * \underbrace{\frac{\partial \hat{y}}{\partial z_1}}_{\text{red}} * \underbrace{\frac{\partial z_1}{\partial w_1}}_{\text{blue}}$$

Repeat this for **every weight in the network** using gradients from later layers